>_ CYBER TECH ACADEMY

Defensive Security & Log Analysis

Build practical defensive skills through system hardening, log review, detection logic, and evidence-based triage.

LEVEL: INTERMEDIATE

Creator: Dhanush | **Edition:** 1.0 | **Cost:** Free

How to use this guide

Read the concepts first, run examples only in a system you own or are authorised to use, complete the lab, then answer the review questions without looking back. Commands and outputs can differ by Linux distribution, software version, and local configuration.

Responsible learning: only test systems for which you have explicit permission. These notes are educational and do not grant permission to access, scan, disrupt, or modify third-party systems.

Learning objectives

- Apply least privilege and basic host-hardening principles.
- Understand common log fields and event timelines.
- Correlate events instead of judging isolated entries.
- Triage alerts using severity, confidence, and business context.
- Document a defensible incident handoff.

Contents

- 1. Defensive security and layered controls
- 2. Logs: structure and reliability
- 3. Reading authentication events
- 4. Search, filter, and correlate
- 5. Alert triage and hardening
- Guided practical lab
- Knowledge check
- Glossary

1. Defensive security and layered controls

Defensive security reduces the likelihood and impact of compromise. Preventive controls attempt to stop events; detective controls identify them; corrective and recovery controls restore safe operation. Strong programmes use layers rather than relying on one tool.

Useful foundations include asset inventory, secure baselines, timely patching, multi-factor authentication, least privilege, network segmentation, endpoint protection, centralised logs, backups, and tested recovery procedures.

Hardening starts by removing unnecessary services and accounts, restricting remote administration, applying updates from trusted repositories, enforcing secure configurations, and monitoring changes. Document exceptions and revisit them regularly.

2. Logs: structure and reliability

A log records an event. Common fields include timestamp, host, process or application, user, source address, event type, result, and message. Field names vary across products. A missing field does not necessarily mean an event did not occur.

Time is crucial for correlation. Systems should use reliable time synchronisation and consistent time zones. Record the source and collection time of evidence, and be alert to clock drift, delayed delivery, duplicated events, and retention gaps.

Logs can contain personal information, tokens, IP addresses, and sensitive business details. Limit access, define retention, protect integrity, and collect only what is justified by policy and law.

3. Reading authentication events

Failed logins can result from mistyped passwords, expired credentials, old services, or malicious guessing. A single failure is weak evidence. Repeated failures followed by a successful login, logins from unusual locations, unfamiliar devices, or activity outside normal patterns may deserve investigation.

Do not assume an IP address maps directly to one person. Shared networks, VPNs, mobile carriers, NAT, and compromised devices can complicate attribution. Correlate multiple signals and state uncertainty clearly.

```
2026-10-01T09:00:00Z host-a sshd: Failed password for demo from 192.0.2.44
2026-10-01T09:01:10Z host-a sshd: Failed password for demo from 192.0.2.44
2026-10-01T09:01:42Z host-a sshd: Accepted publickey for demo from 192.0.2.44
```

The addresses and account above are fictional training data. The sequence suggests repeated failures followed by a successful public-key login, but it does not alone prove compromise. Verify whether the activity was expected and correlate with account, device, and change records.

4. Search, filter, and correlate

On Linux, `grep` can search plain-text logs, while `journalctl` can query the systemd journal on systems that use it. Exact log locations and permissions vary by distribution.

```
grep -i 'failed password' sample-auth.log
grep -E 'Failed|Accepted' sample-auth.log
# On systemd systems, where permitted:
journalctl --since '1 hour ago'
```

Good detection logic includes a time window, a count or sequence condition, relevant context, and an exception strategy. For example, many failures from one source followed by success can be a useful lead, but shared offices and automated systems may create false positives.

Use a structured workflow: preserve the original data, filter a working copy, note the query, record counts and time boundaries, and keep a path back to the original evidence. Never alter source evidence to make analysis easier.

5. Alert triage and hardening

Triage asks: what happened, which asset is involved, what evidence supports the alert, how confident is the assessment, what is the potential impact, and what should happen next? Prioritise by impact and urgency while keeping uncertainty visible.

Containment may include disabling a compromised session, isolating a host, blocking a confirmed malicious indicator, or temporarily restricting an account. Such actions should follow incident procedures because they can interrupt legitimate work. Preserve evidence where possible and coordinate with system owners.

After an event, review root cause, detection gaps, control failures, response delays, and recovery results. Turn lessons into measurable actions with owners and deadlines.

Guided practical lab

Use the following fictional log and write a short triage note.

```
09:00Z user=alex result=failed src=192.0.2.10
09:01Z user=alex result=failed src=192.0.2.10
09:02Z user=alex result=success src=192.0.2.10
09:05Z user=alex action=download file=payroll.csv
09:06Z user=alex mfa=not_recorded
```

- State what is directly observed versus what is only a hypothesis.
- List two benign explanations and two concerning explanations.
- Identify what evidence you would collect next.
- Suggest a proportionate containment option if compromise becomes likely.
- Write a concise handoff with time range, source, confidence, and next action.

Do not declare an account compromised solely from this snippet. Verify whether the login and download were expected, inspect the full audit trail, and check the logging configuration.

Knowledge check

Try to answer these before checking the answer key. Write your reasoning, not just a single word.

1. Why should logs be time-synchronised?

Answer: _____

2. Does a failed login prove an attack?

Answer: _____

3. Why preserve original evidence?

Answer: _____

4. What is a false positive in detection?

Answer: _____

5. What should a triage note contain?

Answer: _____

Answer key and reasoning

1. Consistent timestamps make it possible to reconstruct timelines and correlate events across systems.

2. No. It may be user error or a normal service issue; patterns and context matter.

3. It supports integrity, reproducibility, and later review; analysis should not silently change the source.

4. An alert that indicates suspicious activity when the underlying activity is benign or not the condition described.

5. Observed facts, relevant time range and assets, evidence, confidence, potential impact, actions taken, and next steps.

Glossary

Term	Meaning
Hardening	Reducing unnecessary exposure and insecure configuration.
Telemetry	Data describing system or network activity.
Correlation	Connecting related events across time or sources.
Triage	Initial prioritisation and investigation of an alert.
False positive	Benign activity incorrectly flagged as suspicious.
Containment	Limiting an incident's spread or impact.

Further study

Continue with the next Cyber Tech Academy module, practise in a disposable virtual machine or an explicitly authorised lab, and consult official manuals for the tools you use. Verify commands against your own system before running them.

Cyber Tech Academy • Free learning • Learn the theory. Practise safely. Build real skills.